


# Final Project: multi-agent systems

## Project Guidelines

### Overview

The goal of the project is to put some of the theoretical concepts that we saw together during the course into action in the context of a concrete implementation. Projects are completed in groups of 3 students.

---

## Deliverables

All project deliverables must be submitted right after the final presentation.

### 1. GitHub Repository

A public repo that contains:

- **Source Code:** Complete source code alongside the commit history highlighting the individual contributions of each team member.
- `README.md`: Clear instructions explaining how to set up and execute the code. All required dependencies must be pinned to the exact version used during the

demo (e.g., in a `requirements.txt`), along with explicit guidelines for configuring any necessary API keys or external services.

### 2. Demo video

A 5–10 minute max screen recording demonstrating the execution of your system's main scenarios (submitted as a google drive link)

### 3. Presentation Slides

The slides used during your final presentation (in **English** or **French**). The presentation must include:

- Slides briefly explaining any advanced theoretical concept used that was not covered in class
- Slides detailing the overall system design, technical stack, implementation choices, and experimental results (if any)
- A slide explaining how concepts seen in the course map to the project (if any)
- A Slide detailing *who did what* in the team

---

## Evaluation & Grading

You will be evaluated individually in the following manner:

1. **Intermediate evaluation:** the progress of the proposed design / architecture after 1 week (nothing particular to prepare, oral explanation suffices)
2. **Final Evaluation**
    a. **Clarity of Explanations:** Precision when explaining the theory and design choices
    b. **Quality of the Demo:** Convincing execution of your chosen scenario(s)
    c. **Individual Contribution:** of each member of the team
    d. **Q&A Session:** Individual answers to questions following the presentation
3. **Deliverable Quality**

---

# Project Topics

## Project 1: Axelrod’s Tournament (Deterministic vs. self-adaptive LLM strategy)

- **Description:**
  In game theory, **Axelrod’s tournament** is used to evaluate and compare various strategies in the **Iterated Prisoner's Dilemma**. The goal of this project is to reproduce this tournament with a selection (not necessarily all) of classic deterministic strategies (e.g., *Tit-for-Tat*, *Always Cooperate…*) alongside a dynamic "LLM-guided" strategy

  The LLM agent must integrate a self-adaptive **MAPE-K** loop. You will define at least 3 distinct prompt strategies for the LLM agent (e.g., *Merciless*, *Pragmatic…*). The agent must continuously monitor opponent actions and performance and switch strategies dynamically during a match (the strategy adaptation happens through the MAPE-K loop)

- **Demo Goal:**
  Run a live tournament pitting the adaptive LLM strategy against various deterministic strategies. We want to find out if a self-adaptive LLM-guided strategy can successfully make a comeback if it detects poor performance

---

## Project 2: Multi-Agent Frontier Exploration & Map Merging

- **Description:**
  A team of agents explores an unknown grid world under partial observability. Each agent operates independently and maintains its own incomplete mental map based on local perceptions.

  When two or more agents come within communication range (a certain number of cells for example), they execute a **Map-Merging Protocol** to unify their spatial beliefs into a single shared map. Once maps are merged, the agents run a **Contract Net Protocol (CNP)** (or another negotiation protocol) to dynamically divide remaining unexplored "frontier" regions (unexplored tiles bordering known empty space). Bids are calculated based on proximity and path cost (agents can have different speeds for example), dividing frontiers

between agents to prevent redundant exploration coverage and path collisions.

- **Demo Goal:**
  Comparison of total exploration time required to explore 100% of a large, obstacle-dense environment using **1 isolated agent** versus **3 coordinating agents** using map-merging and negotiation-based task allocation.
- **PS:**

  The term “agent” in the context of this topic refers to a regular deterministic agent **(not an LLM-based agent)**

---

## Project 3: Policy Decision-Making via Value-Based Argumentation Frameworks (VAF)

- **Description:**
  This project builds a policy evaluation platform based on Trevor Bench-Capon’s **Value-Based Argumentation Frameworks (VAF)**. The system models debates over public policy issues (such as public transit development, urban planning, zoning...) across multiple political activists (e.g., *Environmentalist*, *Corporate Lobbyist...*).

  Arguments and attack relations are loaded directly from a structured file or database (e.g., JSON, no natural language extraction required). Each argument is mapped to an underlying social value (e.g., *Economy*, *Environment*, *Public Health*). Each actor agent enforces its own strict preference ordering over these values, creating an actor-specific VAF to compute subjective argument acceptability and preferred extensions.
- **Demo Goal:**
  Demonstrate an interactive policy session where arguments are loaded and processed across different activists. The demo must show how dynamically changing a stakeholder's value preference ordering (e.g., shifting priority from *Economy* over *Environment* to *Environment* over *Economy...*) alters which arguments are accepted or rejected and highlighting any changes to the final outcome of the debate.

---

## Project 4: Multi-Agent Social Choice & Preference Aggregation Engine

- **Description:**
  A population of voters must elect a leader from a set of competing candidates. Each agent possesses a **private utility function** representing its true preference ranking and real number valuation over each candidate. Agents can either vote truthfully according to their real preferences or employ strategic voting tactics (**tactical voting**) to manipulate the outcome when they anticipate their top candidate cannot win.

  An Election Audit mechanism processes these preferences under different voting systems:

  1. **Plurality / First Past the Post (FPTP)**
  2. **Two-Round Runoff Voting**
  3. **Single Transferable Vote (STV) (Ranked-Choice Voting)**
  4. **Approval Voting ("Rank-Free" Voting)**

  The audit engine computes the winning candidate under each rule and measures total **Social Welfare** (total utility). By contrasting the theoretical social optimum against the actual utility delivered by each voting system and quantifies the total **utility Loss** caused by said voting system (with or without tactical voting).

- **Demo Goal:**
  Run a large-scale election scenario with a diverse pool of voters. The demo must quantitatively contrast truthful voting against strategic voting across all four voting rules, demonstrating how changing the election mechanism or injecting tactical voting flips the winning candidate and measurably reduces overall utility.

- **PS:**

  The term "agent" in the context of this topic does refers to a regular deterministic agent **(not an LLM-based agent)**

# Tech Stack

- Free tech stack choice (programming languages, packages, frameworks...)
- UI optional: appreciated but **not required**
- For projects involving LLMs:
  - You don't have to use agentic setups if you judge that they consume tokens heavily (A regular API request is fine)
  - Use lighter or smaller models to avoid surpassing rate limits
  - You can look up some of these providers with free tiers:

| Provider | Details |
|---|---|
| **Google Gemini API** (Google AI Studio) | Free Flash and Flash-Lite models, Pro models are paid only. Limits are shown per project in AI Studio |
| **Groq** | Open-weight models only, on very fast hardware, capped by the number of requests and tokens |
| **OpenRouter** | Many models from one key; filter by "free" or use the `:free` suffix. The free roster changes constantly |
| **Mistral** | Free "Experiment" mode for some Mistral models |
| **Cerebras** | Certain number of free tokens per day |
| **Cloudflare Workers AI** | Certain number of free tokens per day |
| **GitHub Models** | Small monthly allowance. |
| **SambaNova** | Credit for new users |
| **Nebius AI** | Credit for new users |